X-StreamRAG: Explainable Time-Aware Streaming RAG for Real-Time Video Event Detection

[Member 1], [Member 2], [Member 3], [Member 4]
IOE413 Streaming Data Analytics, Group 15 (Topic 7)
[Department, Institute]
[emails]

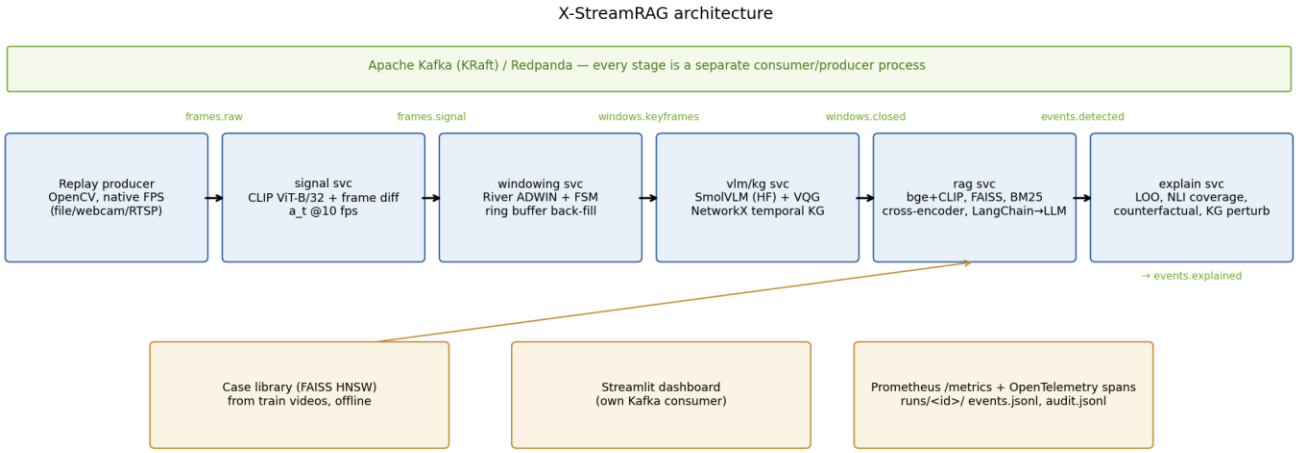


Fig. 1. X-StreamRAG architecture. Each box is a separate process and every arrow is a Kafka topic; Prometheus scrapes every service and the Streamlit dashboard consumes the output topics.

***Abstract*—Retrieval-augmented generation (RAG) over live video can explain what happens in a traffic camera stream, but existing streaming RAG systems decide on single frames or fixed windows and give no evidence for why an alert was raised. We present X-StreamRAG, a streaming pipeline of five independent services that communicate only through Apache Kafka topics. A CLIP and pixel change signal drives River's ADWIN detector and a CALM/ALERT/EVENT state machine that cuts the stream into adaptive windows with a 3 s pre-event back-fill. A small vision-language model describes keyframes into a NetworkX temporal knowledge graph, FAISS, BM25 and a cross-encoder retrieve similar past windows, and a LangChain-orchestrated LLM returns a cited verdict with label-probability confidence. An off-hot-path service measures attribution, grounded coverage and counterfactual evidence removal, and every stage exports Prometheus metrics to a Streamlit dashboard. We re-implement StreamingRAG as a frame-level baseline and compare both systems on 14 traffic clips replayed through Kafka on a CPU-only laptop. The baseline detects more: F1 0.92 (recall 1.00, but both normal clips flagged) against 0.70 for X-StreamRAG (recall 0.58), a difference that is not significant (McNemar $p = 0.22$). X-StreamRAG makes 8× fewer LLM calls, sends 32% fewer keyframes per minute to the VLM and retrieves more relevant evidence (P@5 0.89 vs. 0.58), and removing its top-attributed evidence lowers confidence 2.7–4× more than random removal. We trace every missed detection to a window-count debounce on short clips, and report that the LLM cited no evidence in its explanations.**

***Index Terms*—streaming analytics, retrieval-augmented generation, video event detection, adaptive windowing, explainability, Kafka**

I. Introduction

Traffic surveillance produces continuous, high-velocity video in which incidents such as collisions, rollovers or fires last only a few seconds. Operators need alerts within seconds, and they need to know why an alert was raised before acting on it. Vision-language models (VLMs) and large language models (LLMs) can describe scenes in natural language, but they are far too slow to run on every frame, and black-box alerts are hard to audit.

Retrieval-augmented generation (RAG) grounds an LLM's answer in retrieved evidence [1]. Streaming RAG systems address the cost problem by running a lightweight VLM on selected frames and retrieving context for an LLM [2]. Time-aware RAG shows that retrieving *windows* rather than single observations captures how a situation evolves [3]. Explainability panels for RAG expose attribution, coverage and counterfactual sensitivity [4]. No existing system combines the three ideas for multimodal streams.

A batch approach would struggle here for three reasons. The input is an unbounded stream that must be processed online. Window boundaries must be decided as frames arrive. And the expensive models must be scheduled under a fixed compute budget. We therefore build the pipeline on a real stream backbone (Kafka) with online change detection (River ADWIN).

Problem statement. Given a live traffic video stream, detect safety-critical events (vehicle collision, rollover, fire or explosion) online under a fixed VLM budget, and attach to every

alert evidence whose influence on the decision can be measured. We study four research questions: whether window-level retrieval beats frame-level retrieval (RQ1); whether attribution identifies the evidence that caused a prediction (RQ2); whether adaptive windows save compute without missing short events (RQ3); and whether counterfactual removal of evidence measures how reliable an explanation is (RQ4).

Contributions. (1) An end-to-end Kafka pipeline of five independent services (signal, windowing, VLM/KG, RAG, explain) that reproduces StreamingRAG's core method as a baseline. (2) An enhancement: change-point-driven adaptive windows with pre-event back-fill under a matched VLM budget, with window-level hybrid retrieval and sequence context. (3) An explainability engine for multimodal streaming evidence (E1–E6) that runs off the alert hot path. (4) A quantitative baseline-versus-enhancement comparison that reports negative results honestly.

II. Related Work

StreamingRAG [2] (Sankaradas et al., AI4Sys 2024) extracts real-time context from video with a lightweight VLM. A question bank and a *Visual Query Generator* choose the next questions from the current knowledge-graph state, and an LLM answers standing queries such as "alert on a collision between vehicles and people". Evaluated on intelligent-transportation videos (6 videos, 8 events), it detected 7 of 8 events against 1 of 8 for a caption-every-frame baseline, at about 8 fps. For our purposes its gap is that retrieval and decisions are frame-level, the scheduler is not published, and there is no evidence attribution.

TD-RAG [3] (Xu et al., CyberSciTech 2025) embeds 6 h sliding windows of AirQualityUCI sensor data as text and retrieves the top-5 most similar historical windows under a 90-day recency constraint, reaching 93.3% accuracy against 80.0% for vanilla RAG. Its gap is that windows have a fixed length and the data is a low-rate numeric stream, not video.

CityCopilot-X [4] (Gudipati et al., ICAIC 2026) is a real-time explainability panel for civic text RAG. It offers rewrite, retrieval and rerank attribution, token-level grounded coverage, counterfactual evidence removal (Δconfidence, Δcoverage), drift monitoring and a Prometheus/OpenTelemetry audit bus. In a 24-task pilot it cut post-hoc correction time by 31% and raised citation coverage by 18 points. Its gap is that it works on text documents only.

TABLE I
BASE PAPERS AND WHAT THIS WORK TAKES FROM THEM

Paper	Data / metrics	Taken / gap addressed
StreamingRAG [2]	6 ITS videos; events detected, fps	VLM + question bank + VQG + KG; frame-level only, no attribution
TD-RAG [3]	AirQualityUCI; accuracy	window kNN, top-5, recency; fixed windows, numeric data
CityCopilot-X [4]	24 civic tasks; coverage, correction time	attribution, coverage, counterfactuals; text only

Gap. No prior system adapts its windows to scene changes in a video stream and also measures how faithful its explanations are. We target this gap. Recent work such as StreamRAG [5] also segments streams into events for retrieval, but does not report counterfactual faithfulness.

III. System Architecture

Fig. 1 shows the pipeline. A **replay producer** (OpenCV) reads a video file, webcam or RTSP source and publishes each frame as a JPEG (640 px wide) to the topic `frames.raw`, released at the source's native frame rate. Messages are keyed by `source_id`, and headers carry the frame id, video timestamp and ingest timestamp. Each downstream stage is an independent consumer/producer process with its own consumer group: `frames.raw` → signal → `frames.signal` → windowing → `windows.keyframes` → VLM/KG → `windows.closed` → RAG → `events.detected` → explain → `events.explained`. An end-of-stream marker propagates through all topics so that each stage flushes its per-source state.

Signal service. At about 10 fps it computes the change signal

$$a_t = 0.6\,(1 - \cos(\mathbf{c}_t, \mathbf{c}_{t-1})) + 0.4 \cdot \mathrm{mean}|g_t - g_{t-1}| / 255 \quad (1)$$

where $\mathbf{c}_t$ is the CLIP ViT-B/32 image embedding [6] and g_t a downscaled grey image. The CLIP embedding is forwarded and reused later for visual retrieval.

Windowing service (Sec. IV). It owns the 3 s ring buffer and emits closed windows with their keyframes.

VLM/KG service. SmolVLM-256M-Instruct [7] writes one caption per keyframe and answers up to five questions from a hand-written traffic knowledge base. Yes/no answers are scored as P(yes) from next-token logits. A rule-based Visual Query Generator, Q_{t+k} = VQG(S_t, KB), adds follow-up questions; for example, "two vehicles touching = yes" leads to asking whether a person is lying near the vehicles. Answers become triples in a NetworkX temporal KG with stable IDs (e.g. T17), and nodes older than 120 s are pruned. Every VLM call is cached in SQLite.

RAG service. It embeds the window's text summary with bge-small-en-v1.5 and fuses it with the mean CLIP embedding ($s = 0.7\ \cos_{\text{text}} + 0.3\ \cos_{\text{clip}}$). Candidate evidence comes from a LangChain retriever over a FAISS HNSW case library [8] and a recency-constrained session memory, plus BM25 [9], the current window's caption/VQA units (F1, F2, …) and its KG 1-hop triples (T1, T2, …). A cross-encoder (ms-marco-MiniLM-L-6) keeps the top 8 units. A LangChain prompt → LLM → JSON chain (Qwen2.5-1.5B-Instruct) returns {event_type, start/end, sequence, explanation with [IDs]}. Confidence is never self-reported: we take a softmax over the log-probabilities of the seven label strings. An event opens when P(event) ≥ 0.5 in 2 of the last 3 windows and closes after 2 negative windows.

Explain service (Sec. IV). It consumes `events.detected` off the hot path and writes one audit record per event.

Monitoring. Every service exposes Prometheus metrics (frames ingested, FPS, per-stage latency histograms, VLM calls, end-to-end alert latency, window state) scraped every 1 s, plus OpenTelemetry spans per window. A Streamlit dashboard with its own Kafka consumer shows the live frame, the state badge, the a_t timeline, event cards with clickable evidence IDs, and the explainability panels.

IV. Proposed Enhancement

What is new. Relative to StreamingRAG, we replace fixed-rate frame-level decisions with change-point-driven adaptive windows and window-level hybrid retrieval, and we add a measurable explainability layer. Relative to TD-RAG, windows are adaptive rather than of fixed length, and they are applied to video. Relative to CityCopilot-X, attribution and counterfactuals operate on multimodal streaming evidence: frames, VQA answers, KG triples and past windows. The motivation is that an accident is a short *sequence* (approach, contact, aftermath); a window that follows the scene's own change points should carry that sequence to the LLM while calm stretches cost almost nothing.

Adaptive windows. ADWIN [10] ($\delta = 0.002$) on a_t marks change points that close windows. A state machine enters ALERT when $a_t > \mu + 3\sigma$ over a rolling 30 s window (with an absolute minimum rise of 0.02), when ADWIN fires, or when a hazard is fed back from downstream, and it returns to CALM after 10 s of calm. The CALM, ALERT and EVENT states sample 0.5, 2 and 4 keyframes/s with maximum window lengths of 16, 4 and 2 s. Within each sampling interval the frame with the highest a_t is chosen (priority scheduling). Fig. 2 shows the signal, the windows and the keyframes on one clip.

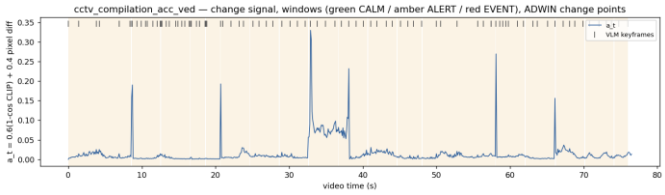


Fig. 2. Change signal a_t on the CCTV compilation clip with B3's adaptive windows (shaded; amber = ALERT) and the keyframes sent to the VLM (ticks). Keyframes concentrate where a_t rises.

Pre-event back-fill. On the CALM → ALERT transition, the two highest-a_t frames of the last 3 s are also sent to the VLM, so a 1 s collision that falls between two CALM samples is still seen.

Fair budget. A token bucket limits B3 to the same average VLM keyframes per minute as the fixed-window variant (60/min), so adaptivity can only redistribute VLM calls, never add more.

Sequence context and hybrid retrieval. The query is the current window plus the previous two, and dense, BM25 and KG candidates are merged and reranked. Every stage's score is logged per evidence unit.

Explainability (E1–E6). E1: leave-one-out attribution $w_i = P(y|U) - P(y|U \setminus \{u_i\})$, in the spirit of ContextCite [11]. E2: the weights are projected on the timeline, giving the top-1 timestamped unit for a temporal pointing game. E3: claim coverage via NLI entailment (DeBERTa-v3-small, $p \geq 0.5$) of each explanation sentence against its cited units, plus token coverage. E4: counterfactual removal of the top-k versus random-k units ($k \in \{1, 2, 3\}$, 5 seeds), reporting Δconfidence, flip rate and the faithfulness gap $\Delta\text{conf}_{\text{top}} - \Delta\text{conf}_{\text{rand}}$. E5: KG triple, entity and relation removal. E6: an embedding-drift monitor over window centroids.

V. Implementation

Stream and data. The public UCF-Crime test videos [12], our proposal's main test set, could not be downloaded on the development laptop. We therefore replay 14 traffic clips: 12 accident clips (2 fixed CCTV, 9 dashcam, 1 news fire) and 2 normal clips, from public GitHub repositories and Intel's CC-BY sample videos. They total 197.5 s of video, or 5,396 frames, published to Kafka at native FPS. Two further normal clips (50 s and 54 s) were dropped to meet the deadline. Labels are per video. The case library (TD-RAG's historical windows) has 84 four-second windows and 309 keyframes from the same 16 clips, each with its video label; at query time, retrieval excludes the query's own video (leave-one-video-out).

Configurations. Table II lists what differs between the two systems. Both use the same VLM, LLM, prompt, knowledge base and event logic. Experiments run in fast-replay mode (timestamps preserved), and LLM feedback to the windowing state machine is disabled for determinism. The explain stage runs after detection because the 16 GB laptop cannot hold two copies of the LLM.

TABLE II
Configurations Compared

Setting	B1 StreamingRAG	B3 X-StreamRAG
Keyframes	fixed 2 fps	adaptive 0.5/2/4 per s
Decision unit	one keyframe (0.5 s)	adaptive window (≤ 16 s)
Retrieval	frame-level kNN	window-level dense + BM25 + KG, rerank
Sequence context	none	current + previous 2
Back-fill	no	3 s ring buffer
Explainability	no	E1–E5

Tools and versions. Table III lists every tool in the pipeline. Hardware: Intel i5-1340P, 16 GB RAM, no GPU. Seeds are fixed, and the configuration, git commit and library versions are saved with every run.

TABLE III
Tools, Versions and Roles

Tool	Version	Role
Apache Kafka (KRaft)	3.9.1	stream backbone (WSL2)
confluent-kafka	2.11.1	Kafka client
River	0.22.0	ADWIN change detection
OpenCV	4.11.0	ingest, JPEG, frame diff
PyTorch (CPU)	2.8.0	model runtime
transformers	4.56.2	SmolVLM, CLIP, Qwen2.5-1.5B
sentence-transformers	5.1.1	bge-small, reranker, NLI
FAISS (cpu)	1.12.0	HNSW case library
rank-bm25	0.2.2	BM25
NetworkX	3.6.1	temporal KG
LangChain core	0.3.76	retriever, prompt chain
Streamlit	1.50.0	dashboard
Prometheus	2.54.1	metrics (1 s scrape)
OpenTelemetry SDK	1.37.0	per-window spans
Python	3.13.5	all services

Design choice. Qwen2.5-0.5B was tried first and rejected because it assigned P(collision) > 0.99 to a calm highway window. With an explicit "default to none" instruction, Qwen2.5-1.5B gives P(none) = 0.96 for that window and P(collision) ≈ 0.99 for a crash window.

Reproducibility. The code, configurations and README are at github.com/Rupsika/streaming_rag. After the setup in the README, `scripts/run_experiment.py --config configs/baselines/b3.yaml` replays the clips through Kafka and writes `results/<run_id>/metrics.json`; `scripts/run_main.sh` runs both systems, the McNemar test and the figures; `scripts/run_all.sh configs/demo.yaml` with `streamlit run dashboard/app.py` starts the live demo.

VI. Experimental Setup and Results

Metrics. Video-level accuracy, precision, recall and F1 (a video is positive if any event is raised), with 95% bootstrap confidence intervals over videos (1,000 resamples) and an exact McNemar test between B1 and B3. We also report the frame-processing ratio, VLM keyframes per minute, window-close-to-verdict latency, throughput, retrieval P@5, and the E1–E5 explainability metrics. Event-level metrics (tIoU ≥ 0.1 or a start within ±2 s of the ground-truth segment, short-event recall, detection delay, pointing game) need annotated event times, which are not yet available for these clips, so they are omitted.

TABLE IV
Main comparison on 14 clips (12 accident, 2 normal)

Metric	B1	B3
Video accuracy	0.857	0.571
Precision / recall	0.857 / 1.000	0.875 / 0.583
F1 [95% CI]	0.923 [0.78, 1.00]	0.700 [0.44, 0.88]
False alarms (normal clips)	2 / 2	1 / 2
Windows (LLM calls)	402	50
VLM keyframes / min	122.1	83.2
Frame-processing ratio	0.074	0.051
LLM s / window (P50)	12.7	98.5
Window latency P50 / P95 (s)	2476 / 4904	1377 / 3851
Retrieval P@5	0.581	0.888

Detection. Table IV and Fig. 3 show that the frame-level baseline detects more accident clips. B1 raised an event on all 12 accident clips (recall 1.00) but also on both normal clips, so its precision is 0.857. B3 raised events on 7 of 12 accident clips (recall 0.583) and on one normal clip (precision 0.875). Per video (Table V), B1 was right on 5 clips where B3 was wrong and B3 on 1 clip where B1 was wrong; the exact McNemar test gives p = 0.22, so with 14 clips neither system is significantly better. Event types were correct on every detected clip for both systems.

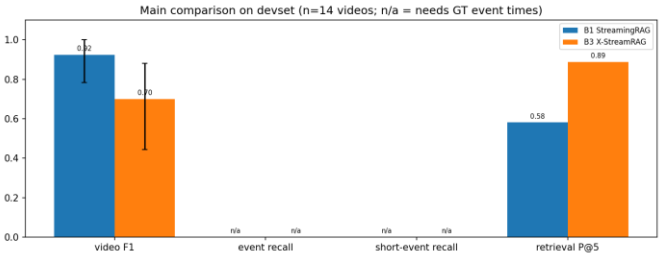


Fig. 3. B1 vs. B3: video F1 with 95% bootstrap CI and retrieval P@5. Event-level metrics are n/a until event times are annotated.

TABLE V
Per-clip windows and decisions (pos/neg = event raised or not; ✓ correct, ✗ wrong)

Clip	GT	s	B1 win.	B1	B3 win.	B3
cctv_compilation	coll.	76.6	154	pos ✓	18	pos ✓
cctv_intersection	coll.	13.2	27	pos ✓	4	pos ✓
motorcycle_1	coll.	5.7	12	pos ✓	2	pos ✓
faizy_traffic	coll.	11.9	24	pos ✓	3	pos ✓
faizy_traffic_2	coll.	17.3	35	pos ✓	5	pos ✓
head_on_10	coll.	5.2	11	pos ✓	1	neg ✗
head_on_109	coll.	3.0	6	pos ✓	1	neg ✗
head_on_114	coll.	6.2	13	pos ✓	1	neg ✗
rollover_1	rollover	5.7	12	pos ✓	1	neg ✗
rollover_106	rollover	4.4	9	pos ✓	1	neg ✗
rollover_121	rollover	5.1	11	pos ✓	2	pos ✓
news_fire_20	fire	5.7	12	pos ✓	2	pos ✓
negative_10	none	7.5	15	pos ✗	1	neg ✓
intel_car	none	30.2	61	pos ✗	8	pos ✗

Compute and latency. B3 groups the same 197.5 s of video into 50 windows instead of 402, i.e. 8× fewer LLM calls, and processes 5.1% of frames with the VLM against 7.4% (83 vs. 122 keyframes/min). Its window latency is lower (P50 1377 s vs. 2476 s, Fig. 4) because far fewer windows queue for the LLM, even though each B3 call is longer (P50 98 s vs. 13 s) since the window prompt carries sequence context and eight reranked evidence units. Both latencies are dominated by queueing in fast-replay mode on a CPU and are not real-time. Window-level hybrid retrieval returns more relevant cases: P@5 rises from 0.58 to 0.89.

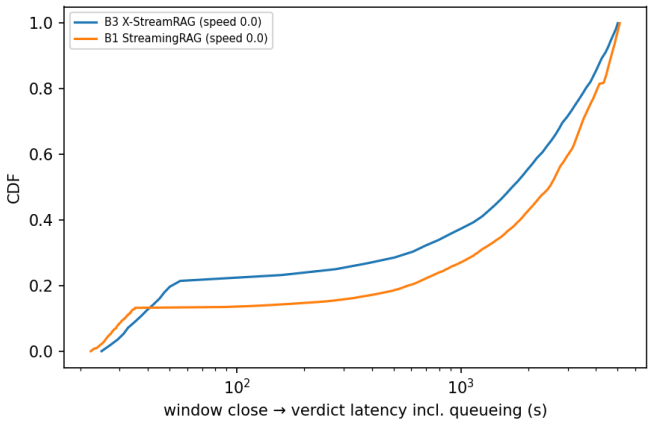


Fig. 4. CDF of window-close-to-verdict latency, including queueing, in fast-replay mode (log scale).

TABLE VI
EXPLAINABILITY OF B3 EVENTS (E1–E5, N = 8)

	k = 1	k = 2	k = 3
Δconf, top-k removed	0.081	0.106	0.114
Δconf, random-k removed	0.020	0.040	0.091
Faithfulness gap	0.061	0.066	0.023
Flip rate top / random	0.25 / 0.23	0.25 / 0.20	0.25 / 0.23
Claim / token coverage	0.00 / 0.00		
Unsupported claims	1.00		

Explainability. Table VI and Fig. 5 cover the 8 B3 events. Removing the top-1 attributed unit lowers confidence by 0.081, four times the drop from removing a random unit (0.020); at k = 2 the ratio is 2.7×. At k = 3 the gap shrinks to 0.023 because random removals start to hit important units too. Flip rates barely differ (0.25 top vs. 0.20–0.23 random), and the absolute drops are small because confidence is saturated (0.97–0.997 on every event). Claim coverage is 0: in all 8 events the LLM wrote a fluent explanation but cited no evidence IDs, so no claim could be checked against its sources.

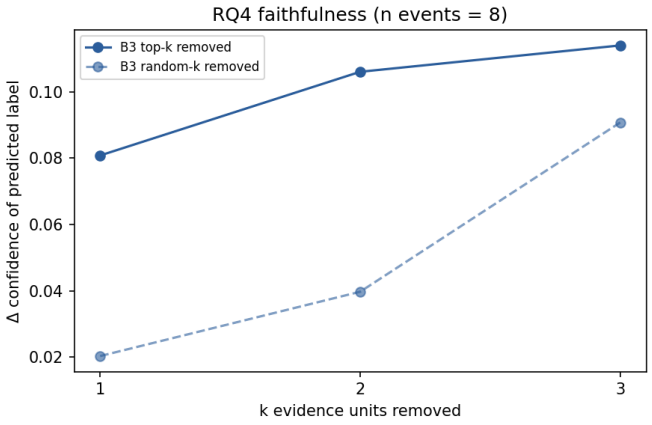


Fig. 5. RQ4: confidence drop when the top-k attributed vs. random-k evidence units are removed.

VII. DISCUSSION AND INFERENCES

Short clips. All five accident clips that B3 missed (3.0–6.2 s long) produced exactly one window (Table V): the change detector stayed in CALM, whose windows may last up to 16 s, so the whole clip became a single window. The LLM scored each of these windows correctly (P(event) = 0.90–0.92, type vehicle collision), but the 2-of-3 debounce needs two positive windows, so no event opened. Every B3 clip with two or more windows was detected. B1 cuts a window every 0.5 s (6–13 windows on the same clips) and never meets this limit, so the shared debounce rule penalises only the adaptive method. The rule should count seconds of positive evidence rather than windows.

The LLM is not discriminating. Every B3 window scored P(event) ≥ 0.5, including those of both normal clips; the normal dashcam clip was classed correctly only because it was also a single window. If single positive windows could open an event, B3 would give 12 TP, 2 FP and 0 TN, exactly B1's result. On this set, therefore, the detection gap between B1 and B3 comes from window counts and the debounce, not from better or worse evidence, and it does not yet show whether adaptive windows help.

False alarms. The 30 s normal parking-lot clip was flagged by B3 in all 8 windows (confidence 0.987). The explanation reads "a collision between vehicles as evidenced by the presence of multiple vehicles and people": the LLM equates a busy scene with a collision. Its strongest attributions are the current window's own captions (0.034 and 0.033), and every unit weighs under 0.04, so no single piece of evidence drove the decision. Together with the saturated confidences, this shows the label-probability confidence is poorly calibrated and 0.5 is far too low a threshold.

Explanations. A positive faithfulness gap at every k means the attribution ranking finds the evidence the model relies on better than chance, so E1 is informative. The effect is small, however, and the flip rates are close to random. The larger problem is grounding: the 1.5B LLM ignored the instruction to cite evidence IDs in all 8 events, so the explanations cannot be audited claim by claim. The pipeline measures this failure instead of hiding it, which is the purpose of the explainability layer.

Answers to the research questions. RQ1: window-level retrieval returns more relevant evidence (P@5 0.89 vs. 0.58), but on this set it did not translate into better detection. RQ2: attribution beats random removal, but the temporal pointing game needs event times and is not yet measured. RQ3: adaptive windows cut VLM keyframes by 32% and LLM calls by 8×, but short clips were missed through the debounce, so the claim that no short events are lost is not supported. RQ4: counterfactual removal does separate attributed from random evidence (gap 0.06–0.07 at $k \leq 2$), though weakly.

Threats to validity. The test set is small (14 clips) and mostly dashcam, while the target is fixed CCTV. The case library comes from the same pool as the test set (leave-one-video-out), which may inflate retrieval quality. Event times are not annotated, so short-event recall (RQ3) and the pointing game (RQ2) are not measured. The fixed-window variant (B2) was not run to

completion, so the effect of adaptivity cannot be separated from that of windowing and hybrid retrieval. B3's measured VLM rate (83 keyframes/min) is above the 60/min target of the token bucket, so the budget match is approximate. Everything runs on a CPU, so latency is far from real time (P50 11.7–14.1 s per VLM keyframe; 12.7 s per B1 and 98.5 s per B3 LLM window; real-time factor 0.05), and the throughput numbers reflect this laptop, not the method.

VIII. Conclusion and Future Work

We built X-StreamRAG, a Kafka-based streaming RAG pipeline that reproduces StreamingRAG as a baseline and extends it with adaptive temporal windows, window-level hybrid retrieval and a measurable explainability layer. On 14 clips it matched the baseline's precision with 8× fewer LLM calls and much better retrieval, but missed five short clips because of a window-count debounce, and its explanations were only weakly faithful and cited no evidence. Future work: calibrating the event threshold, a duration-based debounce, evaluation on UCF-Crime and TAD with annotated event times, the fixed-window and budget-sweep experiments for RQ3, GPU deployment for real-time latency, and a human rating of explanation quality.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[2] M. Sankaradas, R. K. Rajendran, and S. T. Chakradhar, "StreamingRAG: Real-time contextual retrieval and generation framework," in *Proc. AI4Sys*, 2024, arXiv:2501.14101.

[3] Xu, Deng, and Jin, "TD-RAG: Time-aware dynamic-window RAG for risk early warning," in *Proc. IEEE CyberSciTech*, 2025.

[4] Gudipati *et al.*, "CityCopilot-X: A real-time explainability panel for RAG," in *Proc. IEEE ICAIC*, 2026.

[5] Xie *et al.*, "StreamRAG: Enhancing real-time video understanding with retrieval augmentation," in *Proc. CVPR*, 2026.

[6] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021.

[7] A. Marafioti *et al.*, "SmolVLM: Redefining small and efficient multimodal models," arXiv:2504.05299, 2025.

[8] M. Douze *et al.*, "The FAISS library," arXiv:2401.08281, 2024.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, 2009.

[10] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM SDM*, 2007.

[11] B. Cohen-Wang *et al.*, "ContextCite: Attributing model generation to context," in *Proc. NeurIPS*, 2024.

[12] W. Sultani, C. Chen, and M. Shah, "Real-world anomaly detection in surveillance videos," in *Proc. CVPR*, 2018.